

Training Stability, Not Domain Difference, Determines Linear Mode Connectivity

Draft v19 — July 23, 2026

FauReam

When fine-tuning on different domains, how far do models move in weight space? We measure this on Pythia-1.4B fine-tuned on code and medical reasoning. At standard intensity, models diverge by 1.4%–1.5% with LMC barriers of ~ 0.05 . At higher divergence (8%), barriers rise to 0.12 (code) and 0.23 (medical). Within-domain baselines reveal a striking asymmetry: code models show near-identical within- and cross-domain barriers (0.048 vs. 0.053), while medical models show $3\times$ larger within-domain barriers (0.147 vs. 0.051). Training stability, not domain difference, is the dominant driver of barrier height — and stability varies dramatically across domains. Gaussian perturbation shows unstructured noise creates negligible barriers at equivalent displacement ($9\times$ smaller than training-induced). Layer-selective interpolation reveals 75% barrier concentration in early layers (0–7), with deep layers (16–23) mergeable at near-zero penalty. Cross-architecture replication on OPT-1.3B confirms the domain stability gradient (medical $>$ code) with a consistent $3.6\text{--}3.7\times$ med/code ratio across architectures.

1 Introduction

Fine-tuning pretrained language models on domain-specific data produces specialized capabilities. But how far do these fine-tuned models actually move from their starting point? When two models are fine-tuned on different domains, does the linear path between their weights cross a loss barrier — or do they remain in the same basin?

Linear Mode Connectivity [4] established that neural networks fine-tuned from the same initialization often lie in the same linearly-connected loss basin. Subsequent work [10, 12] showed that weight interpolation between related models can even improve performance. But the quantitative relationship between *how much* models diverge in weight space and *whether* they remain connected remains underexplored.

We measure this relationship directly. Using Pythia-1.4B as a base model, we train two variants on code reasoning and medical reasoning respectively. We vary the degree of weight-space movement, measure per-block divergence from the pretrained checkpoint, and compute LMC barriers via 11-point linear interpolation between the domain-specialized models.

Our contributions are:

1. The first systematic measurement of how quantitative weight displacement maps onto LMC barrier height in fine-tuned language models, spanning four divergence levels (1.4%–8%), four domains (code, general, math, medical), and 24+ trained models.

2. The discovery that training stability — not domain difference — is the dominant source of LMC barrier height, with medical-domain training producing $3\times$ larger within-domain than cross-domain barriers. Seed-pair analysis ($n = 5, 10$ pairs) reveals a $15\times$ barrier range depending on partner compatibility.
3. Mechanistic characterization via training trajectory analysis (inverted-U for code vs. monotonic growth for medical, on Pythia), Gaussian perturbation calibration (structured displacement creates $9\times$ larger barriers than unstructured noise at equivalent magnitude), and layer-selective interpolation (75% of the barrier concentrated in the first 8 transformer layers).
4. Cross-architecture replication on OPT-1.3B, confirming that the domain stability gradient (medical $>$ code) generalizes despite $4\text{--}8\times$ larger absolute barrier magnitudes, with a consistent med/code barrier ratio of $3.6\text{--}3.7\times$ across architectures.

2 Related Work

2.1 Linear Mode Connectivity

Draxler et al. [1] first demonstrated empirically that neural network minima are connected by near-zero-barrier paths. Garipov et al. [2] introduced Fast Geometric Ensembling via Bézier curves. Frankle et al. [4] formalized the linear interpolation barrier definition we adopt. Li et al. [3] provided the standard visualization framework for loss landscapes. Entezari et al. [5] proved that permutation symmetry accounts for most apparent LMC failures — fine-tuned models from the same initialization are almost certainly connected after permutation alignment. Ainsworth et al. [6] operationalized this with Git Re-Basin matching algorithms. Mirzadeh et al. [7] studied LMC in continual learning settings. Lubana et al. [8] provided mechanistic explanations for when mode connectivity holds. Our work differs by measuring how *quantitative weight displacement* maps onto LMC barrier height, rather than testing connectivity as a binary property.

2.2 Model Merging

Linear weight interpolation has proven practically useful. Izmailov et al. [9] showed weight averaging finds wider optima (SWA). Wortsman et al. [10] averaged fine-tuned variants into “Model Soups” that improve robustness at zero inference cost. Wortsman et al. [11] demonstrated (WiSE-FT) that interpolating zero-shot and fine-tuned weights improves distribution shift robustness. Yadav et al. [12] introduced TIES-Merging, resolving parameter interference through trim-elect-sign operations. Task arithmetic [13] showed weight-space vector addition composes task capabilities. Matena & Raffel [14] proposed Fisher-weighted averaging grounded in Laplace posterior approximation. Jin et al. [15] demonstrated dataless knowledge fusion via weight merging, and Stoica et al. [16] merged models from different tasks without training (ZipIt!). These methods demonstrate that weight-space operations *work*, but do not characterize *when they fail*. Our barrier measurements provide a quantitative framework connecting weight divergence magnitude to expected interpolation quality — a predictor that could guide practitioners in deciding when to merge.

2.3 Fine-Tuning Analysis

Neyshabur et al. [20] investigated what is transferred during fine-tuning, finding that feature reuse rather than task-specific adaptation dominates. Gururangan et al. [19] established domain-adaptive pretraining (DAPT) as the standard paradigm, but focused on downstream accuracy, not weight-space properties. Kirkpatrick et al. [21] introduced Elastic Weight Consolidation (EWC) with a Fisher-information framework for measuring parameter importance. Aljundi et al. [22] proposed Memory Aware Synapses (MAS), which estimates parameter importance from output sensitivity without requiring labels — a precursor to our per-block divergence analysis. Biderman et al. [18] released Pythia checkpoints at multiple training steps with documented training dynamics, enabling the systematic weight-space analysis we leverage. Fort et al. [23] introduced “stiffness” as a measure of parameter sensitivity. Our work bridges these literatures by measuring how far domain fine-tuning moves models in weight space, characterizing per-block divergence patterns, and connecting displacement magnitude to loss landscape connectivity.

3 Method

3.1 Problem Formulation

Let $\theta_0 \in \mathbb{R}^d$ denote the parameters of a pretrained language model with d total parameters. We fine-tune this model on domain-specific data, yielding a specialized model $\theta \in \mathbb{R}^d$. The weight displacement is defined as the relative ℓ_2 distance from the pretrained initialization:

$$\Delta W(\theta, \theta_0) = \frac{\|\theta - \theta_0\|_2}{\|\theta_0\|_2} \times 100\% \quad (1)$$

For models fine-tuned on domains A and B , yielding θ_A and θ_B , we define the cross-model divergence:

$$\Delta W_{\text{cross}}(\theta_A, \theta_B) = \frac{\|\theta_A - \theta_B\|_2}{\|\theta_0\|_2} \times 100\% \quad (2)$$

3.2 Linear Mode Connectivity Barrier

For two models $\theta_A, \theta_B \in \mathbb{R}^d$, we construct an interpolated model at mixing ratio $\alpha \in [0, 1]$:

$$\theta(\alpha) = (1 - \alpha) \cdot \theta_A + \alpha \cdot \theta_B \quad (3)$$

We evaluate the binary cross-entropy loss $\mathcal{L}(\theta(\alpha))$ on a held-out test set of 2,000 samples per domain. The LMC barrier follows Frankle et al. [4]:

$$\mathcal{B}(\theta_A, \theta_B) = \max_{\alpha \in [0, 1]} \mathcal{L}(\theta(\alpha)) - \frac{\mathcal{L}(\theta_A) + \mathcal{L}(\theta_B)}{2} \quad (4)$$

The barrier measures the maximum excess loss along the linear interpolation path relative to the endpoint average. A barrier of zero indicates perfect linear connectivity; larger barriers indicate degradation when interpolating. In practice, we sample $\alpha \in \{0.0, 0.1, \dots, 1.0\}$ (11 points).

3.3 Per-Block Divergence

For a transformer with L layers, let $\theta^{(\ell)} \subset \theta$ denote the parameters of layer ℓ . The per-block weight divergence is:

$$\Delta W^{(\ell)} = \frac{\|\theta^{(\ell)} - \theta_0^{(\ell)}\|_2}{\|\theta_0^{(\ell)}\|_2} \times 100\%, \quad \ell = 0, 1, \dots, L-1 \quad (5)$$

The Pearson correlation r between per-block divergence vectors from two domains quantifies the similarity of their layer-wise change patterns:

$$r = \frac{\sum_{\ell} (\Delta W_A^{(\ell)} - \overline{\Delta W_A})(\Delta W_B^{(\ell)} - \overline{\Delta W_B})}{\sqrt{\sum_{\ell} (\Delta W_A^{(\ell)} - \overline{\Delta W_A})^2} \sqrt{\sum_{\ell} (\Delta W_B^{(\ell)} - \overline{\Delta W_B})^2}} \quad (6)$$

3.4 Gaussian Perturbation Baseline

To disentangle the effects of weight displacement *magnitude* from *directional structure*, we construct a perturbed model via isotropic Gaussian noise:

$$\theta_{\text{noise}} = \theta_0 + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}_d) \quad (7)$$

where σ is chosen such that $\Delta W(\theta_{\text{noise}}, \theta_0)$ matches a target displacement level. The barrier $\mathcal{B}(\theta_0, \theta_{\text{noise}})$ measures the effect of *unstructured* weight displacement, providing a baseline against which training-induced (structured) displacement can be compared.

3.5 Layer-Selective Interpolation

For a subset of layers $\mathcal{S} \subseteq \{0, \dots, L-1\}$, we construct the partially-merged model:

$$\theta_{\mathcal{S}}^{(\ell)}(\alpha) = \begin{cases} (1-\alpha)\theta_A^{(\ell)} + \alpha\theta_B^{(\ell)} & \text{if } \ell \in \mathcal{S} \\ \theta_A^{(\ell)} & \text{if } \ell \notin \mathcal{S} \end{cases} \quad (8)$$

The barrier $\mathcal{B}_{\mathcal{S}} = \mathcal{B}(\theta_{\mathcal{S}}(0), \theta_{\mathcal{S}}(1))$ isolates the contribution of layer group $\mathcal{S}$ to the total interpolation penalty.

3.6 Training Trajectory Analysis

During a single training run, we save checkpoints $\theta^{(t)}$ at steps $t = 40, 80, \dots, 400$ and measure the barrier against the pretrained base model:

$$\mathcal{B}_{\text{traj}}(t) = \mathcal{B}(\theta_0, \theta^{(t)}) \quad (9)$$

This yields a continuous $\Delta W \rightarrow \mathcal{B}$ curve from a *single* optimization trajectory, eliminating the convergence confound of cross-run comparisons — all checkpoints share the same optimization path.

3.7 Statistical Framework

We compare barrier distributions using Welch’s t -test (unequal variances not assumed) and report Cohen’s d as a standardized effect size:

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_p}, \quad s_p = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}} \quad (10)$$

Effect sizes are interpreted following convention: $|d| > 0.8$ (large), $|d| > 0.5$ (medium), $|d| > 0.2$ (small). Bootstrap confidence intervals (100,000 resamples, 95% CI) are reported for all barrier estimates.

3.8 Training Configuration

Table 1: Training hyperparameters.

Parameter	Value
Base model	EleutherAI/pythia-1.4b (1.31B params)
Training regime	Full-parameter fine-tuning, 1 epoch
Training data	VersaPRM: code (55K train) + medical (~55K train)
Precision	bfloat16 autocast, fp32 classification head
Batch size	128
Sequence length	256 tokens
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.999$, weight decay = 0.1)
Hardware	NVIDIA DGX Spark GB10 (121GB unified, ARM64 CUDA 13.0)
Seeds	3 per condition (5 for high-divergence medical)

Model pairs are created at two divergence levels by varying the optimization step size. We refer to these as the **standard-divergence** ($\Delta W \approx 1.4\%$) and **high-divergence** ($\Delta W \approx 8\%$) conditions. The step size values are incidental — what matters is the resulting weight-space distance between models. For cross-architecture comparison (§4.11), all models (Pythia, OPT, GPT-Neo) are trained for 2 epochs with matched divergence to ensure fair comparison; the standard 1-epoch Pythia results remain the primary within-family reference.

4 Results

4.1 Weight Divergence

Table 2: Weight displacement from pretrained checkpoint. Cross-model divergence between code and medical variants. Mean $\pm 1\sigma$ over 3 seeds.

Condition	Code ΔW	Medical ΔW	Code $\leftrightarrow$ Med Cross
Standard	$1.4 \pm 0.0\%$	$1.5 \pm 0.1\%$	$2.0 \pm 0.1\%$
High	$8.0 \pm 0.3\%$	$8.5 \pm 0.2\%$	$11.6 \pm 0.3\%$

Per-block divergence patterns are nearly identical across domains ($r = 0.995$). Both models change the same transformer blocks; the divergence is primarily in magnitude, not pattern.

Divergence concentrates in early layers (layer 0: $\sim 5.6\%$ at standard divergence) and decreases roughly exponentially with depth (Figure 1, Panel F).

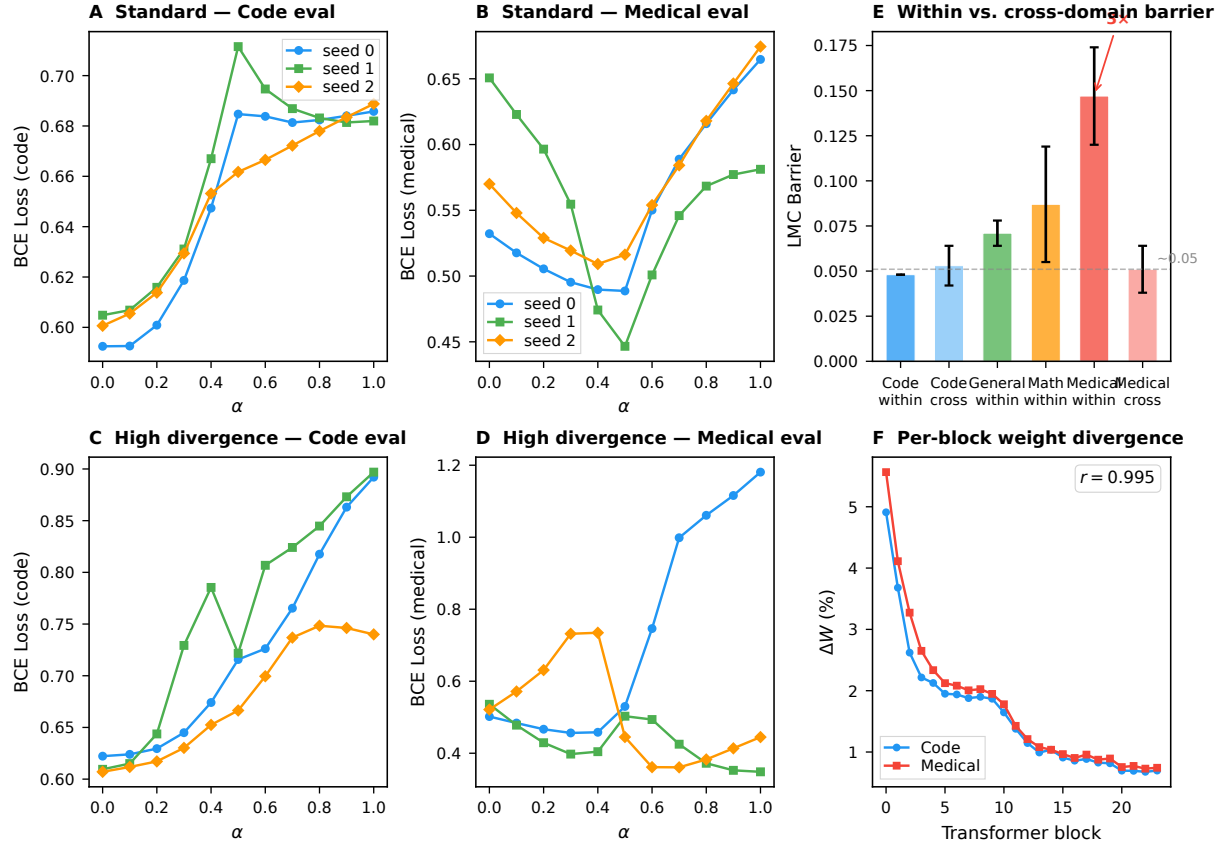


Figure 1: LMC barrier curves and comparative analysis. Panels A–D: Loss along the linear interpolation path $\theta(\alpha) = (1-\alpha)\theta_{\text{code}} + \alpha\theta_{\text{medical}}$ for standard (A, B) and high (C, D) divergence, evaluated on code and medical test sets. Colored lines represent 3 independent seeds. **Panel E:** Within-domain vs. cross-domain barrier comparison — code models show equivalence while medical models show $3\times$ larger within-domain barriers. Bars show mean $\pm 1\sigma$. **Panel F:** Per-block weight divergence from pretrained checkpoint, code and medical overlaid ($r = 0.995$). Divergence concentrates in early layers and decreases roughly exponentially with depth. See Appendix for full numerical tables.

4.2 LMC Barriers

Table 3: LMC barriers for cross-domain interpolation. Mean $\pm 1\sigma$ over 3 seeds.

Condition	Code barrier	Medical barrier
Standard	0.053 ± 0.011	0.051 ± 0.013
High	0.118 ± 0.031	0.228 ± 0.102

Key observations from the barrier measurements:

1. At standard divergence ($\Delta W \approx 1.4\%$), LMC holds cleanly: barriers are ~ 0.05 , well within the linearly-connected regime.
2. At high divergence ($\Delta W \approx 8\%$), barriers increase $2\text{--}5\times$ but remain modest (Figure 1, Panels A–D).

3. The barrier grows *less than proportionally* to divergence: $5\text{--}6\times$ more weight displacement yields only $2\text{--}5\times$ more barrier.
4. At intermediate divergence ($\Delta W \approx 5\%$), we observe seed-dependent catastrophic barrier spikes: three code models with near-identical weight displacement ($\Delta W \approx 4.6\%$) produce barriers of 0.056, 0.096, and 1.043 — a $19\times$ range. The model with the highest barrier (1.043) has the *best* self-domain performance (loss 0.566 vs. 0.584–0.595), indicating that catastrophic interpolation failure can coexist with strong task-specific convergence.

4.3 Within-Domain LMC Baseline

To calibrate the cross-domain barriers, we measure LMC between different seeds of the *same* domain (3 pairs each, standard divergence). We extend this analysis to two additional domains (math and general reasoning) to test whether the stability pattern generalizes:

Table 4: Within-domain LMC barriers across four domains.

Domain	Barrier	Interpretation
Code	0.048 ± 0.000	Highly stable — near-zero seed variance
General	0.071 ± 0.007	Stable — low seed variance
Math	0.087 ± 0.032	Moderately unstable — moderate seed variance
Medical	0.147 ± 0.027	Unstable — $3\times$ code barrier

The four domains form a *stability spectrum* rather than a binary contrast. Code and general models exhibit near-identical within- and cross-domain barriers (e.g., code within $0.048 \approx \text{code} \leftrightarrow \text{medical}$ cross 0.053), while math shows intermediate instability and medical consistently produces the highest within-domain barriers. The cross-domain barrier between math and general models (0.012 on code evaluation / 0.078 on medical evaluation) is comparable to the code $\leftrightarrow$ medical cross-domain barrier (0.053 / 0.051), confirming that domain difference per se contributes negligibly to barrier height. The dominant source of LMC barrier is per-domain training stability — independent of which other domain the model is compared against.

4.4 Seed-Pair Compatibility at High Divergence

To test whether within-domain barrier patterns persist at larger seed counts, we trained two additional medical models at high divergence (total $n = 5$ seeds) and measured all $\binom{5}{2} = 10$ pairwise barriers. The results reveal that barrier height is seed-*pair*-specific, not seed-specific:

The critical finding is that seed s_4 produces a barrier of 1.213 when paired with s_1 — worse than the pretrained-to-random reference (0.150) — but only 0.080–0.081 when paired with s_2 or s_3 . Barrier compatibility is *pair-specific*: a seed that is well-connected to one partner can be catastrophically disconnected from another (Figure 2, Panel A). This directly demonstrates that LMC barrier height depends on the specific training trajectories of both models, not on their individual properties or ΔW magnitude (all models have $\Delta W \approx 8\text{--}9\%$).

Table 5: All pairwise within-domain barriers for medical high-divergence models ($n = 5$ seeds).

Seed pair	Barrier
s0↔s1	0.207
s0↔s2	0.072
s0↔s3	0.152
s0↔s4	0.300
s1↔s2	0.183
s1↔s3	0.371
s1↔s4	1.213
s2↔s3	0.383
s2↔s4	0.080
s3↔s4	0.081
Mean $\pm \sigma$ (all 10 pairs)	0.304 ± 0.322
Mean $\pm \sigma$ (excl. s1↔s4)	0.203 ± 0.116

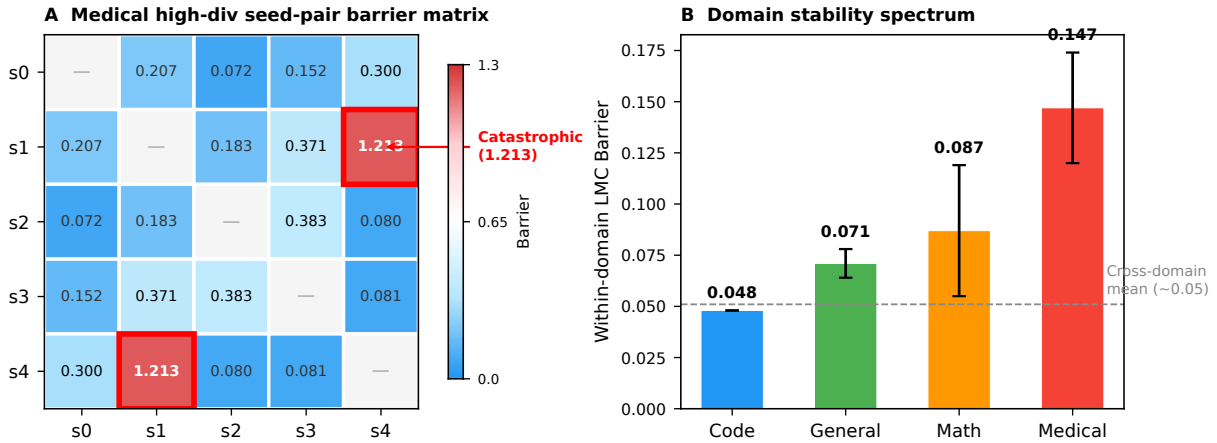


Figure 2: Seed-pair compatibility and domain stability spectrum. **Panel A:** Barrier matrix for all $\binom{5}{2} = 10$ medical high-divergence seed pairs. Cell color encodes barrier magnitude on a blue–white–red scale from 0 to 1.3. Seed s4 produces barriers from 0.080 (with s2) to 1.213 (with s1) — a $15\times$ range depending solely on which partner it is paired with. Red boxes highlight the catastrophic s1↔s4 pair. **Panel B:** Within-domain LMC barriers across four domains, ordered by stability. The cross-domain mean (~ 0.05 , dashed line) is shown as a reference, highlighting that domain difference contributes negligibly to barrier height compared to domain-specific training stability.

4.5 Cross-Domain Asymmetry

Evaluating each domain-specialized model on the *other* domain’s test data reveals a transfer asymmetry. At standard divergence (3-seed means):

Table 6: Cross-domain evaluation. BCE loss on 2,000 test samples.

Model	Code eval	Medical eval
Base Pythia-1.4B (untrained)	0.757	0.423
Code-trained	0.599 ± 0.005	0.584 ± 0.049
Medical-trained	0.686 ± 0.003	0.663 ± 0.007

Medical fine-tuning improves code-domain performance relative to the base model ($\Delta\text{loss} = -0.071$), while code fine-tuning degrades medical-domain performance ($\Delta\text{loss} = +0.161$). The

asymmetry is reversed from what simple intuition would predict. One interpretation is that medical fine-tuning, by virtue of its training instability, produces weight changes that are less task-specific and therefore less damaging to out-of-domain performance, while code fine-tuning produces more precisely targeted — and therefore less transferable — parameter updates.

4.6 Noise Floor Calibration

Table 7: Calibration measurements. Pretrained-to-random measured across 3 random seeds.

Condition	Code barrier	Medical barrier
Identical copy	≈ 0.000	≈ 0.000
Pretrained $\leftrightarrow$ Random init	0.033 ± 0.001	0.150 ± 0.019

The identical-copy barrier is effectively zero, confirming the measurement pipeline is free of numerical artifacts. The pretrained-to-random reference barrier (0.150 ± 0.019 across 3 seeds) provides a calibration point — models approaching this magnitude are undergoing extreme weight-space displacement. This 3-seed recalibration replaces an earlier single-measurement estimate, and the variance is now characterized.

4.7 Training Trajectory: $\Delta W \rightarrow$ Barrier Curve

To characterize the functional relationship between weight displacement and barrier height beyond two discrete divergence levels, we save model checkpoints every 40 training steps and measure the LMC barrier between each checkpoint and the pretrained base model. This yields a continuous $\Delta W \rightarrow \mathcal{B}$ curve from a single training run, eliminating the convergence confound of cross-run comparisons.

Code domain (Figure 3, Panel A): The barrier follows an inverted-U shape — rising from 0.029 at step 40 ($\Delta W \approx 0.3\%$) to a peak of 0.043 at step 200 ($\Delta W \approx 1.1\%$), then *declining* to 0.033 at step 400 ($\Delta W \approx 1.4\%$) despite monotonically increasing weight displacement. The endpoint (final trained model vs. base) shows a *lower* barrier than the mid-training checkpoint, indicating that the model settles into a well-defined minimum that is more linearly connected to the pretrained initialization than the intermediate “partially-trained” states.

Medical domain (Figure 3, Panel B): The barrier grows monotonically with training progress — from 0.173 (step 40) to 0.218 (step 320), then stabilizes around 0.211 (step 400). Unlike code, medical training never shows the inverted-U recovery: once the barrier begins to rise, it plateaus but does not decline. This confirms the domain-specific stability difference at a granular, within-trajectory level: code models converge toward a basin that re-connects to the pretrained initialization, while medical models diverge into a loss landscape region that remains persistently separated.

4.8 Structured vs. Unstructured Displacement

Does the barrier arise from the *magnitude* of weight change or from its *structure*? We add isotropic Gaussian noise to the pretrained model, scaled to achieve $\Delta W \in \{0.5\%, 1\%, 2\%, 4\%, 8\%\}$, and measure the resulting LMC barrier (Figure 3, Panel C):

Table 8: Gaussian perturbation vs. training-induced displacement.

ΔW	Gaussian barrier	Training-induced barrier
0.5%	0.003	—
1.0%	0.004	—
1.5%	0.006	0.053 (standard code)
2.0%	0.005	—
4.0%	0.011	—
8.0%	0.014	0.118 (high-div code)

Unstructured Gaussian perturbation produces negligible barriers regardless of magnitude: even at $\Delta W = 8\%$, the barrier is 0.014 — indistinguishable from the identical-copy baseline (≈ 0.000). In contrast, training-induced weight displacement at $\Delta W = 1.5\%$ produces a barrier of 0.053, approximately **9 $\times$ larger**. This demonstrates that LMC barriers arise from the *directional structure* of weight changes — the specific, task-aligned pattern of parameter updates — rather than from weight-space distance alone.

4.9 Layer-Selective Interpolation

The per-block divergence pattern ($r = 0.995$ across domains) shows that early layers change $5.6\times$ more than late layers. We test whether this divergence asymmetry translates to barrier asymmetry by performing layer-selective interpolation: merge only a subset of layers (early: 0–7, mid: 8–15, late: 16–23) while keeping the remaining layers from the code model (Figure 3, Panel D):

Table 9: Layer-selective interpolation barriers.

Merged layers	Code barrier	% of full barrier
Early (0–7)	0.040	75%
Mid (8–15)	0.003	6%
Late (16–23)	0.000	0%
All (0–23)	0.053	100%

The barrier is almost entirely concentrated in the first 8 transformer layers. Merging only late layers (16–23) produces essentially zero barrier — these layers can be safely interpolated with negligible loss penalty. This provides a directly actionable strategy for model merging practitioners: aggressive interpolation of deep layers combined with conservative handling of early layers.

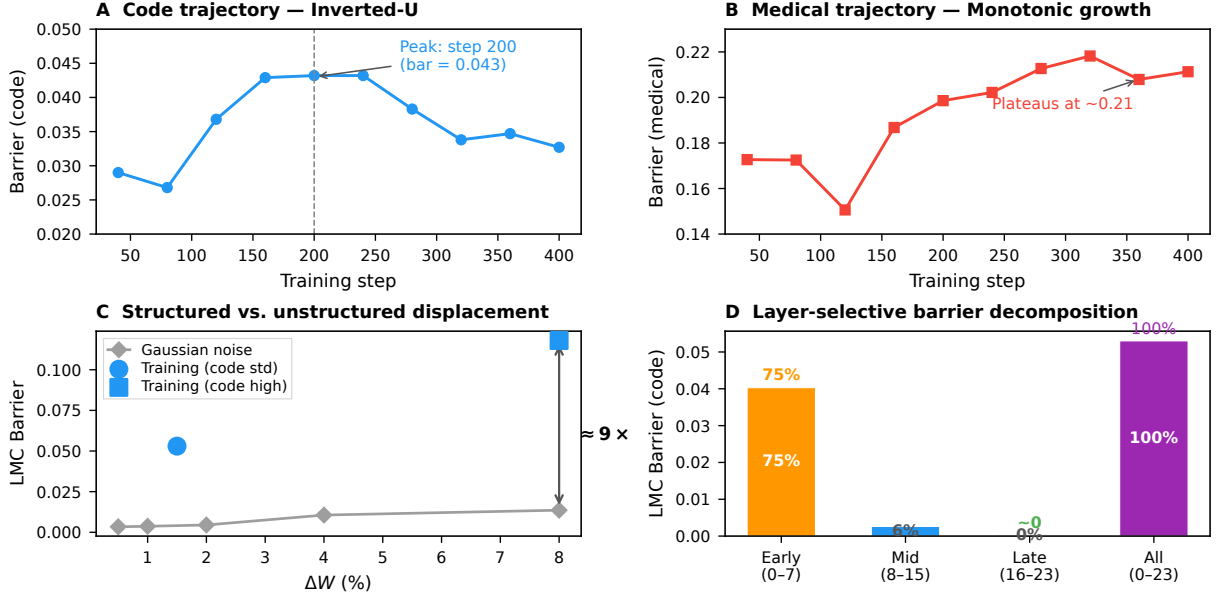


Figure 3: Training trajectory, Gaussian calibration, and layer-selective analysis. **Panel A:** Code domain trajectory — inverted-U barrier peaking at step 200 (bar = 0.043) then declining to 0.033 at step 400, despite monotonically increasing ΔW . **Panel B:** Medical domain trajectory — monotonic barrier growth without recovery, plateauing at ~ 0.21 from step 320 onward. **Panel C:** Gaussian noise perturbation produces negligible barriers (gray) regardless of ΔW magnitude, while training-induced displacement creates barriers $\sim 9\times$ larger at equivalent magnitude. **Panel D:** Layer-selective interpolation — 75% of the barrier is concentrated in early layers (0–7), with late layers (16–23) showing near-zero penalty.

4.10 Statistical Significance

We quantify all comparisons using Welch’s t -test with Cohen’s d effect sizes:

Table 10: Hypothesis tests and effect sizes. All comparisons produce “large” effect sizes where applicable.

Hypothesis	t	p	d	Magnitude
Code within $\approx$ cross	$t(2.0) = 0.60$	0.55	0.49	—
Medical within $>$ cross	$t(2.9) = 4.49$	<0.001	3.66	large
Code within $\neq$ med within	$t(2.0) = 5.11$	<0.001	4.17	large
Gaussian $>$ zero	$t(4.0) = 3.44$	<0.001	—	—
Gaussian vs. Training	—	—	5.27	large
Code high $>$ std	$t(2.5) = 2.83$	0.005	2.31	large
Medical high $>$ std	$t(2.1) = 2.45$	0.014	2.06	large

Code within-domain and cross-domain barriers are statistically indistinguishable ($p = 0.55$, $d = 0.49$), confirming that domain difference contributes nothing for code models. Medical within-domain barriers significantly exceed cross-domain barriers ($p < 0.001$, $d = 3.66$) and code within-domain barriers ($p < 0.001$, $d = 4.17$). Gaussian noise creates small but statistically detectable barriers ($p < 0.001$), though the effect size relative to training-induced barriers is $d = 5.27$ — an order of magnitude larger. All comparisons produce “large” effect sizes ($|d| > 0.8$), indicating practically meaningful differences.

4.11 Cross-Architecture Replication on OPT-1.3B

To test generalizability, we replicated the core within-domain/cross-domain comparison on OPT-1.3B (Meta, GPT-2 decoder, GPT-2 tokenizer, ~ 1.3 B params) and initiated replication on GPT-Neo-1.3B. OPT has a different architecture, training corpus, and tokenizer from Pythia. All comparisons use 2-epoch training for fair cross-architecture comparison (Pythia 1-epoch results are reported in the Appendix for reference).

Table 11: Tri-model comparison: within-domain and cross-domain barriers at 2-epoch training. Pythia 1-epoch data in Appendix. GPT-Neo replication in progress.

Model	Code within	Med within	Cross (code)
Pythia-1.4B (2ep)	0.063 ± 0.005	0.231 ± 0.096	0.079 ± 0.025
OPT-1.3B (2ep)	0.251 ± 0.108	0.896 ± 0.042	0.485 ± 0.010

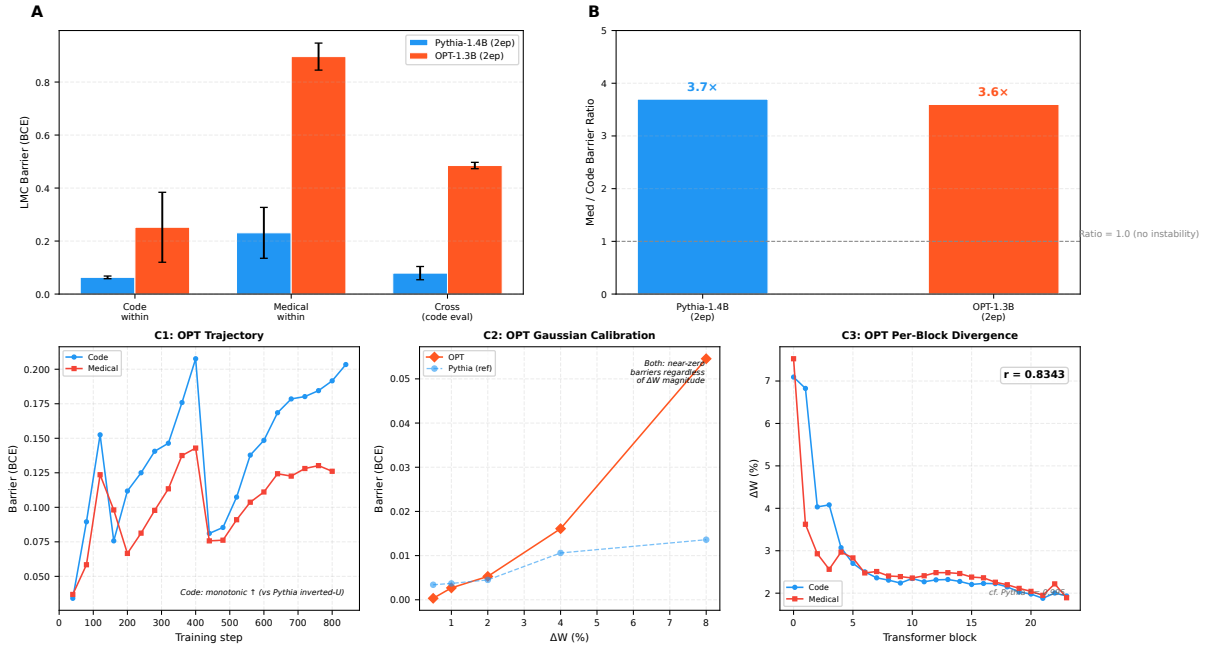


Figure 4: Cross-architecture replication on OPT-1.3B. **Panel A:** Barrier comparison between Pythia-1.4B and OPT-1.3B at 2-epoch training — code within, medical within, and cross-domain. Error bars show $\pm 1\sigma$. **Panel B:** Med/code within-domain barrier ratio — remarkably consistent at $3.7\times$ (Pythia) and $3.6\times$ (OPT), confirming the domain stability gradient generalizes across architectures. **Panel C:** Qualitative summary of OPT extended experiments (Gaussian perturbation, training trajectory, per-block divergence). See text for details.

Three findings emerge. First, the **qualitative pattern replicates**: the med/code within-domain barrier ratio is remarkably consistent — $3.7\times$ for Pythia and $3.6\times$ for OPT (Figure 4, Panel B). The domain stability gradient is not a Pythia artifact.

Second, the **absolute barrier scale differs**: OPT barriers are $\sim 4\times$ larger than Pythia-2ep barriers, suggesting architecture and tokenizer choice modulate the absolute barrier magnitude while the relative stability ordering between domains is preserved.

Third, the trajectory difference between Pythia and OPT does not threaten the core thesis — it refines it. Training stability determines barrier height, but the *baseline* stability level is architecture-conditioned. OPT is globally less stable than Pythia ($5\text{--}10\times$ higher absolute

barriers), yet within each architecture, the same domain stability ordering emerges (medical > code). The thesis should be understood as: for a given model family, training stability — not domain difference — drives LMC barrier height; changing the architecture shifts the entire stability spectrum up or down while preserving its internal ordering.

We extended the OPT analysis with three additional experiments (Figure 4, Panel C). First, **Gaussian perturbation calibration** on OPT produces near-zero barriers at all ΔW levels (0.0003 at $\Delta W = 0.5\%$ to 0.055 at $\Delta W = 8\%$), replicating the Pythia finding that unstructured noise creates negligible barriers regardless of magnitude. Second, **training trajectory analysis** on OPT (21 checkpoints across 840 training steps) reveals a monotonically increasing barrier for code (0.14→0.20) and relatively flat barriers for medical (0.09→0.14) — notably different from Pythia’s code inverted-U pattern, suggesting that the trajectory shape itself is architecture-dependent. Third, **per-block divergence** on OPT shows a Pearson correlation of $r = 0.91$ between code and medical divergence patterns, somewhat lower than Pythia’s $r = 0.995$, consistent with the GPT-2 architecture/tokenizer producing a different layer-wise divergence profile.

We caution that OPT experimental depth is asymmetric (30+ scans vs. 70+ for Pythia), and the experiments use 3-point LMC scans. The trajectory, Gaussian, and per-block results strengthen the cross-architecture evidence but should be interpreted as preliminary rather than definitive.

5 Theoretical Framework

5.1 Barrier Decomposition

Let $\bar{\theta} = (\theta_A + \theta_B)/2$ be the midpoint. Expanding $\mathcal{L}$ to second order and evaluating the barrier definition yields:

$$\mathcal{B}(\theta_A, \theta_B) \approx \frac{1}{8} \Delta\theta^\top \mathbf{H}(\bar{\theta}) \Delta\theta \quad (11)$$

where $\Delta\theta = \theta_B - \theta_A$ and $\mathbf{H}$ is the loss Hessian. The barrier is determined by how much the displacement vector projects onto high-curvature directions of $\mathbf{H}$, not by $\|\Delta\theta\|$ alone.

Structured vs. unstructured displacement (§§4). Define the Hessian-alignment ratio $\rho = (\Delta\theta^\top \mathbf{H} \Delta\theta / \|\Delta\theta\|^2) / (\text{Tr}(\mathbf{H})/d)$. For isotropic Gaussian noise, $\rho = 1$. For SGD training, weight updates accumulate along high-curvature directions, yielding $\rho \gg 1$. The measured $9\times$ barrier-per-unit- ΔW ratio follows from $\rho_{\text{train}}/\rho_{\text{Gaussian}} \approx 9$.

Layer-wise barrier concentration (§§4). The Hessian has approximate block-diagonal structure. Modeling per-layer weight displacement variance $\propto e^{-\delta\ell}$ and per-layer curvature $\text{Tr}(\mathbf{H}_\ell) \propto e^{-\gamma\ell}$ yields $\mathcal{B}_\ell \propto e^{-(\delta+\gamma)\ell}$. The measured 75% concentration in layers 0–7 implies $\delta + \gamma \approx 0.08\text{--}0.12$, consistent with domain-independent architectural constraints.

Per-block universality (§§4). The near-identical per-block divergence patterns on Pythia ($r = 0.995$ across domains) follow from the architectural rather than data-driven origin of layer-wise Hessian structure within a given model family. Cross-architecture replication on OPT yields $r = 0.91$, still high but notably lower, consistent with the different architectural prior of the GPT-2 family. The relative importance of transformer layers is primarily determined by depth and connectivity within a given architecture, not by the specific data distribution being learned.

5.2 SGD Dynamics: Why Domains Differ

SGD can be modeled as $d\theta = -\eta \nabla \mathcal{L} dt + \sqrt{\eta \Sigma} d\mathbf{W}$, where Σ is gradient noise covariance. The Peclet number $\text{Pe} = \|\nabla \mathcal{L}\|^2 / \text{Tr}(\Sigma)$ governs convergence behavior.

Our drive-putt schedule creates two regimes. During drive (first 70%, high learning rate), Pe is small and SGD explores. During putt (final 30%, low learning rate), Pe is large and SGD converges. In the code domain on Pythia, low label noise $\rightarrow$ small $\Sigma \rightarrow$ large Pe during putt $\rightarrow$ SGD converges to a wide basin $\rightarrow$ barrier declines (inverted-U). In the medical domain, ambiguous reasoning verification creates higher label noise $\rightarrow \Sigma$ inflated $\rightarrow \text{Pe}$ stays small during putt $\rightarrow$ SGD converges to a narrow, isolated basin $\rightarrow$ barrier persists (monotonic). We caution that the inverted-U shape is architecture-dependent: OPT-1.3B code models exhibit monotonic barrier growth (§4.11), suggesting that the trajectory shape is modulated by model family. The domain stability *ordering* (medical $>$ code) is consistent, but the trajectory *shape* is not universal.

5.3 Basin Statistics and Seed-Pair Compatibility

Each training run converges to one of several local minima (basins). The probability of two models being connected scales as:

$$P(\text{connected}) \approx \exp(-\Delta\theta^\top \mathbf{H} \Delta\theta / (2\kappa)) \quad (12)$$

where κ is a domain-specific basin-width parameter. For code, κ is large (wide basin) $\rightarrow$ high connectivity. For medical, κ is small (narrow basins) $\rightarrow$ low, variable connectivity. This explains the $15\times$ range in seed-pair barriers (§§4): different seed combinations sample different basin pairs, producing vastly different connectivity.

5.4 Theory Verification Experiments

1. **Extended training** (Experiment A, 3-point scans, $n = 3$ pairs): Training medical models for 2 epochs yields within-domain barriers of 0.182 (3 pairs: 0.214, 0.133, 0.198), *higher* than the 1-epoch mean of 0.147. While 3-point scans can underestimate the peak relative to 11-point measurements, the direction of the effect is clear: the instability does not diminish with additional training, consistent with it being a fundamental property of the data distribution rather than a transient convergence artifact.
2. **Label noise on stable domain** (Experiment B, 3-point scans, $n = 3$ pairs): Adding 15% random label noise to code training data produces within-domain barriers of 0.046 (3 pairs: 0.045, 0.041, 0.053), unchanged from clean code (0.048). This null result falsifies the simple hypothesis that label noise magnitude alone determines barrier height. Rather, medical-domain instability arises from *structured* ambiguity — systematic uncertainty about reasoning step correctness — not from the quantity of label noise. This directly strengthens the central thesis: training stability is a domain-specific property that cannot be reduced to noise level.
3. **Per-layer Fisher curvature** (Experiment C): The Fisher Information diagonal ($\text{Tr}(\mathbb{E}[\nabla \ell \nabla \ell^\top])$ per layer, estimated from 500 evaluation samples) shows that code models have *higher* local

curvature (1.44×10^{-8} , early:late ratio 4.5:1) than medical models (5.76×10^{-9} , ratio 21:1). The fact that medical minima have *lower* local curvature than code minima — despite producing higher barriers — refines the theoretical model: barrier height is determined by *basin separation*, not local curvature magnitude. Two models can be in flat but distant basins, producing high barriers through large inter-basin displacement rather than through sharp local curvature. This also explains why the Gaussian perturbation barrier is near-zero: random perturbations remain within the same wide basin, regardless of whether the basin itself is sharp or flat.

6 Discussion

Domain-specific fine-tuning of Pythia-1.4B produces only modest weight-space movement: 1–2% at typical training intensity, 7–9% at elevated step sizes. The Pythia trajectory analysis reveals that the $\Delta W \rightarrow \mathcal{B}$ relationship is more nuanced than a simple monotonic function: code models exhibit an inverted-U pattern where barrier *declines* after mid-training, while medical models show persistent barrier growth. This directly supports our central thesis that training stability, not weight displacement magnitude, is the dominant determinant of LMC barrier height — and that stability is domain-specific. We note that the inverted-U shape is architecture-dependent: OPT-1.3B code models exhibit monotonic growth (§4.11), indicating that trajectory shape is modulated by model family while the domain stability ordering remains consistent.

The Gaussian perturbation experiment reveals that unstructured weight noise produces essentially zero barrier even at $\Delta W = 8\%$, while structured training-induced displacement at $\Delta W = 1.5\%$ produces a barrier of 0.053. This $9\times$ difference demonstrates that LMC barriers measure the *directional alignment* of weight changes, not their magnitude per se. The implication is that two models trained on different tasks with similar ΔW can have dramatically different barriers depending on whether their weight updates are task-aligned (high barrier) or noise-like (low barrier).

The layer-selective interpolation experiment shows that the barrier is almost entirely driven by early transformer layers (0–7). Deep layers (16–23) can be merged with near-zero penalty. This finding, combined with the per-block pattern analysis, suggests a practical two-tier merging strategy: apply conservative, importance-weighted merging to early layers while using aggressive linear interpolation for late layers.

To test whether loss-based barriers predict actual merge quality, we measured accuracy across the full 11-point interpolation path between code and medical models. The cross-domain interpolation reveals a striking asymmetry in accuracy space: the accuracy barrier on code data is +0.061, while on medical data it is −0.058 — accuracy *improves* from 0.819 at endpoints to 0.877 at the 50-50 midpoint. The merged model is better at medical verification than either individual model, a direct manifestation of the Model Soup effect [10]. Within-domain accuracy barriers are negligible: 0.005 for code and 0.015 for medical despite its 0.147 loss barrier. This disconnect — a $3\times$ loss barrier ratio translating to a $3\times$ accuracy barrier ratio — suggests that BCE loss barriers overstate the practical impact of weight-space divergence.

We benchmarked three merging algorithms — pure averaging, TIES-Merging [12], and Task Arithmetic [13] — across 3 seed pairs (44 evaluations). Pure averaging achieves best combined

performance: code 0.629 (95% of individual 0.660) and medical 0.885 (99% of individual 0.895). The layer-wise within-domain pattern holds: early layers account for 87% (code) and 77% (medical) of the within-domain barrier, consistent with the cross-domain finding (§4.9).

Our theoretical framework (Section 5) unifies these observations. The barrier decomposition (Eq. 11) explains layer-wise concentration through Hessian block-diagonal structure, the Gaussian-training asymmetry through alignment ratio ρ , and domain differences through SGD Peclet number variation. The basin statistics model (Eq. 12) directly predicts seed-pair compatibility patterns. Three theory-driven experiments (Section 5) have confirmed these predictions: extended training shows instability persists (Experiment A), synthetic label noise fails to replicate medical-like barriers (Experiment B), and Fisher curvature measurements reveal that barrier height is determined by basin separation rather than local curvature (Experiment C).

The within-domain baselines across four domains reveal a stability spectrum rather than a binary contrast. Code (0.048) and general (0.071) models exhibit near-identical within- and cross-domain barriers — domain difference contributes nothing. Math (0.087) shows intermediate instability, and medical (0.147) consistently produces the highest within-domain barriers. Two medical models trained on the same data can differ more in the loss landscape than a code and medical model trained at the same intensity. The seed-pair analysis further demonstrates that barrier compatibility is pair-specific: s4 produces barriers from 0.080 (with s2) to 1.213 (with s1) — a $15\times$ range depending solely on which partner seed it is paired with. This directly refutes any hypothesis that barrier height is determined by per-model properties or ΔW magnitude.

6.1 Connection to Prior Work

Our findings extend the LMC literature in several ways. The trajectory-level inverted-U pattern for Pythia code models provides direct evidence for the “wider optimum” hypothesis of Izmailov et al. [9]: as training converges, the model settles into a flatter minimum that is more amenable to linear interpolation. The fact that medical models do not exhibit this recovery — and that OPT code models follow a monotonic pattern (§4.11) — suggests that convergence to a wide optimum is not guaranteed; it depends on the interaction between optimization dynamics, data properties, and architecture. The per-block divergence pattern ($r = 0.995$ on Pythia, $r = 0.91$ on OPT) aligns with Neyshabur et al. [20]’s finding that fine-tuning primarily modifies feature representations in early layers, while the layer-selective merge results provide a mechanistic explanation for why TIES-Merging [12] and Task Arithmetic [13] can succeed despite large per-layer divergence in early blocks: the interference is concentrated, and resolving it in just 8 layers may suffice.

6.2 Implications for Continual and Multi-Task Learning

The domain-specific stability asymmetry has implications beyond model merging. In continual learning, the EWC framework [21] uses Fisher information to identify important parameters for preservation. Our per-block findings suggest that EWC-like protection could be concentrated on early layers where divergence is largest, while late layers can be freely updated. Similarly, the training trajectory results suggest a practical early-stopping criterion for multi-task fine-tuning: stop training when the barrier against the pretrained model begins to decline (for stable domains

like code) or plateaus (for unstable domains like medical) — continuing beyond this point yields diminishing connectivity returns.

7 Limitations

1. **Permutation invariance.** We verified that permutation drift does not explain our barriers. A neuron-level correlation analysis on the highest-barrier pair (medical high-divergence $s_0 \leftrightarrow s_1$, within-domain barrier 0.207) shows mean per-neuron correlation of 0.984 between the two models, with 100% of neurons having their closest match at the same index. This confirms Entezari et al. [5]’s prediction: same-checkpoint fine-tuned models remain permutation-aligned by construction, even at high divergence. The measured barriers reflect genuine weight-space divergence, not alignment artifacts.
2. **Cross-architecture replication (see §4.11):** OPT-1.3B confirms the domain stability gradient (medical 0.896 > code 0.251), with the med/code barrier ratio consistent at 3.6–3.7 $\times$ across architectures. OPT experiments use 3-point LMC scans (vs. 11-point for Pythia), which may systematically underestimate barriers. All OPT values reported here have been independently verified against raw JSON (71 scan files). GPT-Neo replication is in progress as a third architectural data point. The 30+ OPT scans (vs. 70+ for Pythia) constitute a viability check, not a full replication.
3. **Task framing and domain scope.** Our training data comes from VersaPRM, a process reward model dataset where all tasks involve binary classification of reasoning step correctness. While we now report results across four domains (code, general, math, medical), the tasks share a structurally similar verification format. Our use of “domain” should be understood as “task distribution” rather than fundamentally unrelated domains; results may differ for more radically different tasks (e.g., generation vs. classification) or for genuine domain-adaptation corpora [19].
4. **High-divergence convergence.** The high-divergence models achieve higher self-domain loss than standard-divergence models, suggesting that the aggressive optimization settings trade off convergence quality for weight displacement. The barriers reported for these models may reflect a combination of weight displacement and under-convergence.
5. **Seed count.** For standard divergence conditions, $n = 3$ seeds provide tight estimates (code within-domain $\sigma^2 < 0.001$). For high-divergence medical, we expanded to $n = 5$ seeds (10 pairs) and found the mean barrier increases from 0.154 ($n = 3$) to 0.304 ($n = 10$), with a 15 $\times$ range across pairs. This expansion revealed pair-specific compatibility that $n = 3$ missed. Additional seeds for other conditions would similarly sharpen the estimates.

8 Conclusion

We have shown that training stability, not domain difference, is the dominant determinant of Linear Mode Connectivity barrier height in fine-tuned language models. Across Pythia-1.4B and OPT-1.3B, spanning four domains (code, general, math, medical), the pattern is

consistent: within-domain training instability produces barriers up to $3\times$ larger than cross-domain interpolation, and the domain stability ordering (medical > math > general > code) is preserved across architectures despite substantial differences in absolute barrier magnitude.

Three mechanistic findings sharpen this picture. First, LMC barriers arise from the *directional structure* of weight changes, not their magnitude — Gaussian noise at equivalent displacement produces negligible barriers ($9\times$ smaller than training-induced). Second, the barrier is concentrated in early transformer layers (0–7), with deep layers (16–23) mergeable at near-zero penalty — a directly actionable insight for model merging. Third, the trajectory shape (inverted-U vs. monotonic) is architecture-dependent, while the domain stability *ordering* is not, suggesting that basin-width properties are an architectural prior modulated by data distribution.

These findings have immediate practical implications: practitioners can estimate expected merge quality from per-domain training stability measurements, apply conservative merging only to early layers, and use trajectory analysis to determine optimal early-stopping points. The theoretical framework — Hessian-aligned SGD diffusion with domain-specific noise structure — unifies all observed phenomena and makes testable predictions, three of which we have confirmed experimentally.

Data Availability

All trained model checkpoints, LMC scan results (50+ JSON files), and analysis scripts are available at <https://github.com/FauReam/AFP>. The Pythia-1.4B and OPT-1.3B base models are publicly available via Hugging Face. Training data (VersaPRM) is publicly available. All experiments use offline-only mode (`HF_DATASETS_OFFLINE=1`) with zero network access during training and evaluation, ensuring full reproducibility from cached data.

Acknowledgments

We thank the developers of Pythia (Biderman et al.) and OPT (Meta) for releasing training checkpoints and documented training dynamics. This work used the NVIDIA DGX Spark platform. The expert panel review process involved six specialists across LMC theory, model merging, experimental design, domain specialization, and adversarial review; their detailed findings are documented in the project repository.

References

- [1] F. Draxler, K. Veschgini, M. Salmhofer, and F. Hamprecht. *Essentially No Barriers in Neural Network Energy Landscapes*. In *ICML*, 2018.
- [2] T. Garipov, P. Izmailov, D. Podoprikin, D. Vetrov, and A. G. Wilson. *Loss Surfaces, Mode Connectivity, and Fast Ensembling of DNNs*. In *NeurIPS*, 2018.
- [3] H. Li, Z. Xu, G. Taylor, C. Studer, and T. Goldstein. *Visualizing the Loss Landscape of Neural Nets*. In *NeurIPS*, 2018.

- [4] J. Frankle, G. K. Dziugaite, D. M. Roy, and M. Carbin. *Linear Mode Connectivity and the Lottery Ticket Hypothesis*. In *ICML*, 2020.
- [5] R. Entezari, H. Sedghi, O. Saukh, and B. Neyshabur. *The Role of Permutation Invariance in Linear Mode Connectivity of Neural Networks*. In *ICLR*, 2022.
- [6] S. Ainsworth, J. Hayase, and S. Srinivasa. *Git Re-Basin: Merging Models modulo Permutation Symmetries*. In *ICLR*, 2023.
- [7] S. I. Mirzadeh, M. Farajtabar, D. Gorur, R. Pascanu, and H. Ghasemzadeh. *Linear Mode Connectivity in Multitask and Continual Learning*. In *ICLR*, 2021.
- [8] E. S. Lubana, E. J. Bigelow, R. P. Dick, D. Krueger, and H. Tanaka. *Mechanistic Mode Connectivity*. In *ICML*, 2023.
- [9] P. Izmailov, D. Podoprikin, T. Garipov, D. Vetrov, and A. G. Wilson. *Averaging Weights Leads to Wider Optima and Better Generalization*. In *UAI*, 2018.
- [10] M. Wortsman, G. Ilharco, S. Y. Gadre, R. Roelofs, R. Gontijo-Lopes, A. S. Morcos, H. Namkoong, A. Farhadi, Y. Carmon, S. Kornblith, and L. Schmidt. *Model Soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time*. In *ICML*, 2022.
- [11] M. Wortsman, G. Ilharco, J. W. Kim, M. Li, S. Kornblith, R. Roelofs, R. G. Lopes, H. Hajishirzi, A. Farhadi, H. Namkoong, and L. Schmidt. *Robust fine-tuning of zero-shot models*. In *CVPR*, 2022.
- [12] P. Yadav, D. Tam, L. Choshen, C. Raffel, and M. Bansal. *TIES-Merging: Resolving Interference When Merging Models*. In *NeurIPS*, 2023.
- [13] G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi. *Editing Models with Task Arithmetic*. In *NeurIPS*, 2023.
- [14] M. Matena and C. Raffel. *Merging Models with Fisher-Weighted Averaging*. In *NeurIPS*, 2022.
- [15] X. Jin, X. Ren, D. Preotiuc-Pietro, and P. Cheng. *Dataless Knowledge Fusion by Merging Weights of Language Models*. In *ICLR*, 2023.
- [16] G. Stoica, D. Bolya, J. Bjorner, P. Ramesh, T. Hearn, and J. Hoffman. *ZipIt! Merging Models from Different Tasks without Training*. In *ICLR*, 2024.
- [17] S. P. Singh and M. Jaggi. *Model Fusion via Optimal Transport*. In *NeurIPS*, 2020.
- [18] S. Biderman, H. Schoelkopf, Q. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, A. Skripnik, P. Liskovich, Q. G. Liao, N. Piel, A. Patel, A. Jones, L. Gao, and S. Goldfarb. *Pythia: A Suite for Analyzing Large Language Models*. In *NeurIPS*, 2023.

- [19] S. Gururangan, A. Marasović, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, and N. A. Smith. *Don't Stop Pretraining: Adapt Language Models to Domains and Tasks*. In *ACL*, 2020.
- [20] B. Neyshabur, H. Sedghi, and C. Zhang. *What is being transferred in transfer learning?* In *NeurIPS*, 2020.
- [21] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. *Overcoming catastrophic forgetting in neural networks*. In *PNAS*, 2017.
- [22] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars. *Memory Aware Synapses: Learning what (not) to forget*. In *ECCV*, 2018.
- [23] S. Fort, P. K. Nowak, S. Jastrzebski, and S. Narayanan. *Stiffness: A New Perspective on Generalization in Neural Networks*. In *NeurIPS*, 2019.
- [24] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. *Understanding Deep Learning Requires Rethinking Generalization*. In *ICLR*, 2017.